

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport"
content="width=device-width,initial-scale=1,maximum-scale=1,user-scalable=no">

<title>Paper Sketch AR</title>

<style>
html,body{
  margin:0;
  height:100%;
  overflow:hidden;
  background:#000;
  font-family:Arial;
  touch-action:none;
}

#v{
  position:fixed;
  inset:0;
  width:100%;
  height:100%;
  object-fit:cover;
}

#o{
  position:fixed;
  left:50%;
  top:50%;
  transform:translate(-50%,-50%);
  max-width:90vw;
  max-height:80vh;
  touch-action:none;
  transform-origin:center center;
}

#bar{
  position:fixed;
  bottom:0;
  left:0;
  right:0;
  padding:8px;
```

```
background:#000d;  
color:#fff;  
z-index:10;  
text-align:center;  
}
```

```
button,label{  
padding:8px;  
margin:2px;  
border:0;  
border-radius:8px;  
background:#fff;  
font-weight:bold;  
}
```

```
input[type=range]{  
width:80px;  
}
```

```
#pick{  
display:none;  
}
```

```
#msg{  
position:fixed;  
top:8px;  
left:8px;  
right:8px;  
text-align:center;  
color:#fff;  
background:#0009;  
padding:7px;  
border-radius:8px;  
z-index:20;  
}
```

```
#brush{  
position:fixed;  
z-index:15;  
display:none;  
pointer-events:none;  
border:2px solid red;  
border-radius:50%;  
transform:translate(-50%,-50%);
```

```

}
</style>
</head>

<body>

<video id="v" autoplay playsinline muted></video>

<img id="o" hidden>

<div id="msg">
 Camera →  Photo →  Sketch Overlay
</div>

<div id="brush"></div>

<div id="bar">

<button id="start"> Camera</button>

<button id="capture"> Take Photo</button>

<label>
 Gallery
<input id="pick" type="file" accept="image/*">
</label>

<button id="lock"> Lock</button>
<button id="reset"> Reset</button>

<br>

Opacity
<input id="op" type="range" min="0" max="100" value="65">

Dark
<input id="dk" type="range" min="50" max="220" value="150">

Zoom
<input id="zm" type="range" min="30" max="300" value="100">

<br>

 Brush

```

```
<input id="bs" type="range" min="1" max="50" value="5">
```



```
<input id="color" type="color" value="#000000">
```

```
</div>
```

```
<script>
```

```
const v=document.getElementById('v');  
const o=document.getElementById('o');  
const pick=document.getElementById('pick');
```

```
const start=document.getElementById('start');  
const capture=document.getElementById('capture');  
const lock=document.getElementById('lock');  
const reset=document.getElementById('reset');
```

```
const op=document.getElementById('op');  
const dk=document.getElementById('dk');  
const zm=document.getElementById('zm');
```

```
const bs=document.getElementById('bs');  
const color=document.getElementById('color');  
const msg=document.getElementById('msg');  
const brush=document.getElementById('brush');
```

```
let stream=null;  
let locked=false;
```

```
let s=1;  
let x=0;  
let y=0;
```

```
let drag=false;  
let sx=0;  
let sy=0;  
let ox=0;  
let oy=0;
```

```
let pinch=false;  
let startDistance=0;  
let startScale=1;
```

```

let brushSize=5;
let brushColor='#000000';

/* CAMERA */

start.onclick=async()=>{

  try{

    stream=await navigator.mediaDevices.getUserMedia({
      video:{
        facingMode:{ideal:'environment'}
      },
      audio:false
    });

    v.srcObject=stream;

    msg.textContent=
      ' LIVE CAMERA — paper पर reference sketch align करें';

  }catch(e){

    alert(
      'Camera permission दें। HTTPS page जरूरी है।'
    );

  }

};

/* TAKE PHOTO FROM CAMERA */

capture.onclick=()=>{

  if(!stream){

    alert('पहले Camera Start करें।');
    return;

  }

}

```

```
const canvas=document.createElement('canvas');

canvas.width=v.videoWidth || 1280;
canvas.height=v.videoHeight || 720;

const ctx=canvas.getContext('2d');

ctx.drawImage(
  v,
  0,
  0,
  canvas.width,
  canvas.height
);

processImage(canvas);

};
```

```
/* GALLERY */
```

```
pick.onChange=e=>{

  const f=e.target.files[0];

  if(!f)return;

  const im=new Image();

  im.onload={()=>{

    const canvas=document.createElement('canvas');
    const ctx=canvas.getContext('2d');

    const max=1200;

    const z=Math.min(
      1,
      max/Math.max(im.width,im.height)
    );

    canvas.width=im.width*z;
    canvas.height=im.height*z;
```

```
    ctx.drawImage(
        im,
        0,
        0,
        canvas.width,
        canvas.height
    );

    processImage(canvas);

};

im.src=URL.createObjectURL(f);

};

/* MAKE SKETCH */

function processImage(canvas){

    const ctx=canvas.getContext('2d');

    const w=canvas.width;
    const h=canvas.height;

    const data=ctx.getImageData(
        0,
        0,
        w,
        h
    );

    const a=data.data;

    const gray=new Float32Array(w*h);

    let i,p;

    for(
        i=0,p=0;
        i<a.length;
        i+=4,p++
```

```

){

    gray[p]=
        .299*a[i]+
        .587*a[i+1]+
        .114*a[i+2];

}

const out=ctx.createImageData(w,h);

for(
    let yy=1;
    yy<h-1;
    yy++
){

    for(
        let xx=1;
        xx<w-1;
        xx++
    ){

        p=yy*w+xx;

        const gx=
            -gray[p-w-1]
            -2*gray[p-1]
            -gray[p+w-1]
            +gray[p-w+1]
            +2*gray[p+1]
            +gray[p+w+1];

        const gy=
            -gray[p-w-1]
            -2*gray[p-w]
            -gray[p-w+1]
            +gray[p+w-1]
            +2*gray[p+w]
            +gray[p+w+1];

        const edge=
            Math.min(
                255,

```

```

        Math.hypot(gx,gy)
    );

    const k=p*4;

    out.data[k]=
    out.data[k+1]=
    out.data[k+2]=
        255-edge;

    out.data[k+3]=255;

}

}

ctx.putImageData(out,0,0);

o.src=canvas.toDataURL('image/png');

o.hidden=false;

x=0;
y=0;
s=1;

zm.value=100;

apply();

msg.textContent=
    '🖍️ SKETCH OVERLAY — 1 finger move, 2 fingers zoom';

}

/* APPLY */

function apply(){

    o.style.opacity=
        op.value/100;

    o.style.filter=

```

```
    'contrast('+  
    dk.value+  
    '%) brightness('+  
    (260-dk.value)+  
    '%)';  
  
    o.style.transform=  
    'translate(calc(-50% + '+  
    x+  
    'px),calc(-50% + '+  
    y+  
    'px)) scale('+  
    s+  
    ')';  
}
```

```
/* CONTROLS */
```

```
op.oninput=apply;
```

```
dk.oninput=apply;
```

```
zm.oninput=(()=>{
```

```
    s=zm.value/100;
```

```
    apply();
```

```
});
```

```
/* ONE FINGER MOVE */
```

```
o.addEventListener(
```

```
    'pointerdown',
```

```
    e=>{
```

```
        if(!locked)return;
```

```
        drag=true;
```

```
        sx=e.clientX;
```

```
    sy=e.clientY;  
  
    ox=x;  
    oy=y;  
  
    o.setPointerCapture(e.pointerId);  
  
    }  
);
```

```
o.addEventListener(  
  'pointermove',  
  e=>{  
  
    if(  
      drag &&  
      !locked  
    ){  
  
      x=  
        ox+  
        e.clientX-  
        sx;  
  
      y=  
        oy+  
        e.clientY-  
        sy;  
  
      apply();  
  
    }  
  
  }  
);
```

```
o.addEventListener(  
  'pointerup',  
  ()=>{  
    drag=false;  
  }  
);
```

```
/* TWO FINGER PINCH ZOOM */
```

```
o.addEventListener(  
  'touchstart',  
  e=>{  
  
    if(!locked)return;  
  
    if(e.touches.length===2){  
  
      pinch=true;  
  
      startDistance=  
        distance(  
          e.touches[0],  
          e.touches[1]  
        );  
  
      startScale=s;  
  
    }  
  
  },  
  {passive:false}  
);
```

```
o.addEventListener(  
  'touchmove',  
  e=>{  
  
    if(  
      locked ||  
      e.touches.length!==2 ||  
      !pinch  
    )return;  
  
    e.preventDefault();  
  
    const d=  
      distance(  
        e.touches[0],
```

```

        e.touches[1]
    );

    s=
    startScale*
    (d/startDistance);

    s=Math.max(
        0.3,
        Math.min(3,s)
    );

    zm.value=s*100;

    apply();

},
{passive:false}
);

o.addEventListener(
    'touchend',
    e=>{

        if(e.touches.length<2){

            pinch=false;

        }

    }
);

function distance(a,b){

    return Math.hypot(
        a.clientX-b.clientX,
        a.clientY-b.clientY
    );

}

```

```
/* LOCK */
```

```
lock.onclick={()=>{
```

```
    locked=!locked;
```

```
    lock.textContent=
```

```
        locked
```

```
        ? '🔒 Locked'
```

```
        : '🔓 Lock';
```

```
    msg.textContent=
```

```
        locked
```

```
        ? '🔒 LOCKED — अब paper पर trace करें'
```

```
        : 'Overlay unlocked';
```

```
};
```

```
/* RESET */
```

```
reset.onclick={()=>{
```

```
    x=0;
```

```
    y=0;
```

```
    s=1;
```

```
    zm.value=100;
```

```
    op.value=65;
```

```
    dk.value=150;
```

```
    apply();
```

```
};
```

```
/* BRUSH */
```

```
bs.oninput={()=>{
```

```
    brushSize=
```

```
        Number(bs.value);
```

```
brush.style.width=
brushSize+'px';

brush.style.height=
brushSize+'px';

};

color.oninput=(()=>{

brushColor=
color.value;

brush.style.borderColor=
brushColor;

});

/* BRUSH POINTER */

document.addEventListener(
'pointermove',
e=>{

if(
e.pointerType==='pen' ||
e.buttons
){

brush.style.display='block';

brush.style.left=
e.clientX+'px';

brush.style.top=
e.clientY+'px';

}

}
);
```

</script>

</body>

</html>